

FRONTEND · BACKEND · DATABASES · DEVOPS

Master Full Stack Developer Interview Questions

235+ curated questions with concise, interview-ready answers

Compiled & Curated by

Ajeet Yadav

July 2026

Ajeet Yadav

Table of Contents

1. HTML & Accessibility	15 questions
2. CSS & Responsive Design	15 questions
3. JavaScript Core	25 questions
4. TypeScript	10 questions
5. React	25 questions
6. State Management & Data Fetching	10 questions
7. Node.js & Backend Runtime	15 questions
8. REST, GraphQL & API Design	15 questions
9. Databases — SQL	15 questions
10. Databases — NoSQL	10 questions
11. Authentication & Security	15 questions
12. System Design	15 questions
13. DevOps, CI/CD & Cloud	15 questions
14. Testing	10 questions
15. Performance & Web Vitals	10 questions
16. Git & Collaboration	5 questions
17. Behavioral & Scenario	10 questions

Q1

What is semantic HTML and why does it matter?

Semantic HTML uses tags that describe meaning (`<article>`, `<nav>`, `<header>`, `<main>`, `<aside>`) rather than presentation. It improves accessibility for assistive tech, boosts SEO by helping crawlers understand structure, and makes markup easier to maintain.

Q2

Difference between `<section>`, `<article>`, and `<div>`.

`<article>` is self-contained, independently distributable content (blog post, card). `<section>` groups related content with a heading. `<div>` is a generic non-semantic container used only when no semantic tag fits.

Q3

Explain the purpose of the DOCTYPE declaration.

`<!DOCTYPE html>` tells the browser to render the page in standards mode instead of quirks mode, ensuring consistent modern behavior.

Q4

What are data-* attributes?

Custom attributes that store extra info on elements without polluting the DOM. Read them via `element.dataset.key` or `[data-key]` in CSS.

Q5

How do you make an image accessible?

Provide meaningful alt text describing the image's purpose. Use empty `alt=""` for decorative images so screen readers skip them. Use `<figure>/<figcaption>` for captions.

Q6

What is ARIA and when should you use it?

Accessible Rich Internet Applications provides roles, states and properties for assistive tech. Use it only when native HTML cannot express the semantics — 'no ARIA is better than bad ARIA.'

Q7

Difference between id and class.

id is unique per document and used for anchors, labels, or JS lookups. class is reusable and used for styling groups of elements. IDs have higher CSS specificity.

Q8

What are void/self-closing elements?

Elements that cannot have children — `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`. In HTML they need no closing tag; in XHTML/JSPX they self-close as `<br/>`.

Q9**Explain the difference between defer and async on <script>.**

async downloads in parallel and executes as soon as ready, blocking parsing briefly. defer downloads in parallel but waits until HTML parsing finishes and executes in order. Prefer defer for most app scripts.

Q10**What is the difference between <link rel='preload'> and 'prefetch'?**

preload fetches critical resources needed for the current page with high priority. prefetch fetches resources likely needed for future navigations at low priority.

Q11**How do form validation attributes work?**

required, minlength, maxlength, pattern, type=email/url/number, and min/max provide native validation. The browser blocks submission and shows messages; :invalid/:valid pseudo-classes enable styling.

Q12**What is the tabindex attribute?**

Controls keyboard focus order. tabindex='0' includes an element in natural order, '-1' makes it focusable only via script, positive values force order (avoid — hurts accessibility).

Q13**Difference between <button> and <a>.**

<button> triggers actions. <a href> navigates to a URL. Using the wrong one breaks keyboard/screen-reader semantics; never use <div onclick>.

Q14**What are landmarks in accessibility?**

Regions like header, nav, main, aside, footer that let screen-reader users jump around the page. Use one <main> per page.

Q15**What are essential meta tags for SEO?**

<title> (<60 chars), <meta name='description'> (<160 chars), viewport, canonical, Open Graph (og:title, og:image), and Twitter Card tags. Add JSON-LD structured data when applicable.

Q1
6**Explain the CSS box model.**

Every element is a rectangle of content + padding + border + margin. `box-sizing: border-box` includes padding and border inside the declared width, which is usually easier to reason about.

Q1
7**Difference between relative, absolute, fixed, and sticky positioning.**

relative offsets from itself but keeps space. absolute removes from flow and positions relative to nearest positioned ancestor. fixed positions relative to viewport. sticky toggles between relative and fixed based on scroll position.

Q1
8**How does CSS specificity work?**

Specificity is (inline, ids, classes/attrs/pseudo-classes, elements). Higher wins; ties go to source order. !important overrides but should be avoided.

Q1
9**Flexbox vs Grid — when to use each?**

Flexbox is one-dimensional (row OR column) and content-driven — best for nav bars, toolbars, form rows. Grid is two-dimensional and layout-driven — best for page layouts and complex responsive designs.

Q2
0**What are media queries?**

CSS conditions that apply rules based on device features like width, orientation, `prefers-color-scheme`. e.g. `@media (min-width: 768px) { ... }`. Foundation of responsive design.

Q2
1**Explain rem, em, vh, vw, %.**

rem is relative to the root font-size; em to the parent's font-size. vh/vw are 1% of viewport height/width. % is relative to the parent's corresponding dimension.

Q2
2**What are CSS custom properties (variables)?**

`--name: value` declared on a selector, consumed via `var(--name)`. They cascade, can be overridden per scope, and be updated at runtime via JS — essential for theming.

Q2
3**How does z-index work?**

Controls stacking order among positioned elements within the same stacking context. New stacking contexts are created by `opacity < 1`, `transform`, `filter`, `position + z-index`, etc.

Q2
4

Difference between visibility:hidden, display:none, opacity:0.

display:none removes from layout. visibility:hidden keeps layout but hides and disables interaction. opacity:0 hides but still interactive and takes space.

Q2
5

What is the difference between pseudo-classes and pseudo-elements?

Pseudo-classes (:hover, :focus, :nth-child) target states. Pseudo-elements (::before, ::after, ::placeholder) target parts of an element. Single vs double colon convention.

Q2
6

Explain CSS transitions vs animations.

Transitions animate between two states triggered by a change. @keyframes animations run defined sequences on load or on demand, with control over iterations, direction, and timing.

Q2
7

What is a stacking context?

An isolated set of layers where z-index is meaningful. Created by root element, positioned+z-index, opacity<1, transforms, filters, will-change, etc.

Q2
8

How does mobile-first design work?

Write base styles for small screens, then use min-width media queries to progressively enhance for larger screens. Produces smaller CSS on mobile and cleaner overrides.

Q2
9

What are logical properties?

margin-inline-start, padding-block-end, etc. — properties that adapt to writing direction (LTR/RTL, vertical). Preferred for internationalized sites.

Q3
0

How do you optimize CSS for performance?

Minify, remove unused rules, avoid deep selectors, use will-change judiciously, prefer transform/opacity for animation (GPU accelerated), and inline critical CSS.

Q3
1**Explain var vs let vs const.**

var is function-scoped and hoisted with undefined. let and const are block-scoped and hoisted into a temporal dead zone. const prevents reassignment (but object contents are still mutable).

Q3
2**What is a closure?**

A function bundled with references to its lexical scope. It lets an inner function access outer variables even after the outer function returns — foundation for private state, currying, and module patterns.

Q3
3**Explain the event loop.**

JS runs on a single thread. Sync code runs on the call stack. Async callbacks queue into the macrotask queue (setTimeout, I/O) or microtask queue (promises, queueMicrotask). After the stack empties, all microtasks run before the next macrotask.

Q3
4**Difference between == and ===.**

== does type coercion before comparing. === compares value and type without coercion. Always prefer === to avoid surprising conversions.

Q3
5**What is hoisting?**

The JS engine allocates declarations before executing code. Function declarations hoist fully; var hoists as undefined; let/const hoist into TDZ and throw if accessed early.

Q3
6**Explain 'this' in JavaScript.**

'this' depends on how a function is called: global/undefined in strict mode, the object before the dot for methods, explicit via call/apply/bind, or the lexical 'this' for arrow functions.

Q3
7**What are Promises?**

Objects representing the eventual result of an async operation. States: pending, fulfilled, rejected. Chain via .then/.catch/.finally; combine with Promise.all/allSettled/race/any.

Q3
8**Explain async/await.**

Syntactic sugar over promises. async functions always return a promise; await pauses execution until a promise settles. Use try/catch for errors.

Q3
9

Difference between call, apply, and bind.

call(thisArg, arg1, arg2) and apply(thisArg, [args]) invoke immediately with a set 'this'. bind returns a new function with 'this' (and optionally args) pre-set.

Q4
0

What is prototypal inheritance?

Objects link to a prototype via [[Prototype]]. Property lookup walks the chain until found or null. class syntax is sugar over prototypes.

Q4
1

Deep vs shallow copy.

Shallow copies (spread, Object.assign) duplicate top-level values only — nested references are shared. Deep copies (structuredClone, lodash cloneDeep) recursively duplicate.

Q4
2

Explain debounce vs throttle.

Debounce delays execution until N ms after the last call — good for search input. Throttle limits execution to once per N ms — good for scroll/resize handlers.

Q4
3

What are Map and Set?

Map is a keyed collection with any key type and preserved insertion order. Set stores unique values. Both offer O(1) add/has/delete.

Q4
4

Explain the difference between null and undefined.

undefined means a variable is declared but not assigned. null is an intentional 'no value' assignment. typeof null is 'object' (legacy bug).

Q4
5

What is destructuring?

Syntax to unpack values from arrays/objects into variables: const {a, b=1} = obj; const [x, y] = arr;. Supports defaults, renaming, and rest.

Q4
6

Difference between forEach, map, filter, reduce.

forEach iterates for side effects. map returns a new array of transformed items. filter returns a new array of items passing a predicate. reduce folds an array into a single value.

Q4
7

What are generators?

Functions declared with function* that can pause via yield and be resumed. Used for lazy sequences, coroutines, and custom iterators.

Q4
8

Explain Symbol.

A unique, immutable primitive often used as object keys to avoid collisions. Well-known symbols like `Symbol.iterator` customize built-in behavior.

Q4
9

What is the difference between spread and rest operators?

Both use `...` syntax. Spread expands iterables into args/elements. Rest collects remaining args/elements into an array.

Q5
0

Explain event delegation.

Attach one listener to a common ancestor and use `event.target` to identify which child triggered it. Reduces memory and handles dynamically added elements.

Q5
1

What are Web Workers?

Background threads that run JS off the main thread. Communicate via `postMessage`. Useful for CPU-heavy work without blocking the UI.

Q5
2

Explain memoization.

Caching a function's return based on its inputs so repeated calls with the same args are instant. Trade memory for CPU.

Q5
3

What is the temporal dead zone?

The period between entering a scope and the actual `let/const` declaration during which access throws `ReferenceError`.

Q5
4

Difference between microtasks and macrotasks.

Microtasks (Promise callbacks, `queueMicrotask`, `MutationObserver`) run after the current task and before rendering. Macrotasks (`setTimeout`, `setInterval`, I/O) run one per event loop tick.

Q5
5

What is optional chaining and nullish coalescing?

`?.` safely accesses deep properties, returning undefined if any link is null/undefined. `??` returns the right side only when the left is null or undefined (unlike `||` which falls back on any falsy value).

Q5
6**What are the benefits of TypeScript over JavaScript?**

Static typing catches bugs at compile time, better IDE autocompletion and refactoring, self-documenting code, and safer large-scale refactors.

Q5
7**Difference between type and interface.**

Both describe object shapes. interface supports declaration merging and classical extends; type supports unions, intersections, mapped/conditional types. Prefer interface for public APIs, type for computed shapes.

Q5
8**What are generics?**

Type parameters that let functions/classes work with multiple types while preserving type info: function identity<T>(x: T): T { return x; }.

Q5
9**Explain unknown vs any vs never.**

any disables type checking. unknown is safe any — must be narrowed before use. never represents unreachable code or values that never occur (e.g. exhaustive switch).

Q6
0**What are utility types?**

Built-in helpers: Partial<T>, Required<T>, Readonly<T>, Pick<T,K>, Omit<T,K>, Record<K,V>, ReturnType<F>, Awaited<T>, etc. Enable reusable type transformations.

Q6
1**Explain type narrowing.**

Reducing a variable's type within a branch via typeof, instanceof, in, equality checks, or user-defined type predicates. TS uses control flow analysis to refine types.

Q6
2**What are conditional types?**

T extends U ? X : Y — types that depend on other types. Combined with infer, they power utility types like ReturnType.

Q6
3**What is a discriminated union?**

A union of object types sharing a literal 'tag' field (kind, type). Switch on the tag to safely narrow — the compiler enforces exhaustiveness.

Q6
4

Difference between interface extends and type intersection.

interface B extends A creates a new interface adding members. type C = A & B intersects; conflicting members become never. Interfaces error on incompatible extends; intersections silently combine.

Q6
5

What are decorators?

Functions that modify classes, methods, properties, or parameters via @syntax. Common in frameworks like NestJS and Angular. Standardized in ES stage 3.

Q6
6**What is the virtual DOM?**

An in-memory representation of the real DOM. React diffs old vs new virtual trees and applies the minimal set of real DOM mutations, reducing costly reflows.

Q6
7**Class vs functional components.**

Functional components are plain functions using hooks for state/effects — the modern default. Class components use lifecycle methods and `this.state`; still supported but no new features are added.

Q6
8**Explain `useState`.**

Hook that returns a stateful value and a setter. Setters trigger re-renders; use functional updater (`prev => next`) when the next state depends on the previous.

Q6
9**Explain `useEffect`.**

Runs side effects after render. The dependency array controls when it re-runs; empty array means once on mount. Return a cleanup function to run on unmount or before re-run.

Q7
0**`useMemo` vs `useCallback`.**

`useMemo` caches a computed value; `useCallback` caches a function reference. Both take a dependency array. Use only when profiling shows real cost — premature memoization hurts.

Q7
1**What is the key prop and why does it matter?**

A stable, unique identifier for list items. React uses keys to match old and new elements, preserving state and avoiding needless remounts. Don't use array index if the list can reorder.

Q7
2**Controlled vs uncontrolled components.**

Controlled inputs derive value from state and update via `onChange`. Uncontrolled inputs manage their own DOM state, read via `refs`. Prefer controlled for validation and complex forms.

Q7
3**What is the Context API?**

Passes data through the component tree without prop drilling. Provider sets value; consumers `useContext`. Best for low-frequency global data like theme, user, locale.

**Q7
4**

What are React refs?

References to DOM nodes or mutable values that persist across renders without triggering re-render. Created with `useRef`; attached via `ref` prop.

**Q7
5**

Explain React reconciliation.

The diffing algorithm that compares the new virtual tree to the previous one and computes the minimal DOM updates. Uses element type and key to decide reuse vs recreate.

**Q7
6**

What is React.memo?

HOC that memoizes a functional component, skipping re-render when props are shallow-equal. Useful for expensive pure components rendered many times.

**Q7
7**

What is a custom hook?

A function starting with 'use' that composes other hooks. Encapsulates reusable stateful logic (`useAuth`, `useDebounce`). Each usage gets isolated state.

**Q7
8**

Explain useReducer.

Alternative to `useState` for complex state or when the next state depends on multiple actions. Takes `(state, action) => newState` — Redux-style.

**Q7
9**

What is React Suspense?

Component that shows a fallback while children are waiting for async data or code. Works with `lazy()`, and with data-fetching libraries that support it.

**Q8
0**

What is code splitting?

Splitting the JS bundle into chunks loaded on demand. In React, use `React.lazy` + `Suspense`, or `dynamic import()` for route-based splitting.

**Q8
1**

What are Error Boundaries?

Class components implementing `componentDidCatch/getDerivedStateFromError` that catch render/lifecycle errors in their subtree and display a fallback. They do NOT catch async or event-handler errors.

**Q8
2**

What is React StrictMode?

Dev-only wrapper that double-invokes render/effects to surface side-effect bugs, warns about deprecated APIs, and helps prepare for concurrent features.

**Q8
3**

SSR vs CSR vs SSG.

CSR renders in the browser (fast to build, slow first paint). SSR renders on the server per request (fresh, good SEO). SSG pre-renders at build time (fastest, cached at CDN).

**Q8
4**

What are React Portals?

Render children into a DOM node outside the parent hierarchy while keeping React tree semantics. Ideal for modals, tooltips, dropdowns to escape overflow/z-index issues.

**Q8
5**

How do you avoid prop drilling?

Extract shared state into Context, use a state library (Zustand, Redux, Jotai), or compose components so state lives closer to where it's used.

**Q8
6**

Explain useEffect vs useLayoutEffect.

useEffect runs asynchronously after paint. useLayoutEffect runs synchronously after DOM mutations, before paint — use only when you must read layout and mutate DOM to avoid a flash.

**Q8
7**

What are Server Components?

React components that run only on the server, streaming HTML/serialized props to the client. Zero client bundle for their code, direct access to server resources.

**Q8
8**

How does React batch state updates?

Multiple setState calls within the same event/tick are batched into one re-render. React 18 extends this to promises, setTimeout, and native handlers (automatic batching).

**Q8
9**

What are keys of a good component design?

Single responsibility, small surface (few props), presentational vs container split, avoid deep prop drilling, and lift state only as high as necessary.

**Q9
0**

How do you optimize a slow React app?

Profile with React DevTools, virtualize long lists (react-window), memoize expensive components, code-split routes, defer non-critical work, and reduce context churn.

Q9
1**When do you need a state library over Context?**

When state is frequent-updating, deeply shared, or needs middleware/devtools. Context re-renders every consumer on change; libraries offer selectors to subscribe to slices.

Q9
2**Redux core concepts.**

Single store, actions describe changes, reducers are pure functions producing next state, dispatch triggers updates, selectors read state. Redux Toolkit is the recommended modern API.

Q9
3**Redux Toolkit vs classic Redux.**

RTK reduces boilerplate with createSlice (reducers + actions in one), createAsyncThunk, and configureStore preconfigured with Immer and devtools.

Q9
4**Zustand vs Redux.**

Zustand is minimal, hook-based, no providers, no boilerplate. Redux is more structured with middleware ecosystem. Choose Zustand for smaller apps, Redux for large teams needing conventions.

Q9
5**What problem does React Query solve?**

Server state — caching, deduping, background refetching, stale-while-revalidate, pagination, and mutations — without hand-rolling loading/error/refetch logic.

Q9
6**Explain stale-while-revalidate.**

Serve cached data immediately (stale), then refetch in the background and update UI when fresh data arrives. Best of both instant UX and freshness.

Q9
7**What is optimistic UI?**

Update UI immediately assuming a mutation will succeed; roll back on failure. React Query and SWR both provide primitives for this.

Q9
8**How do React Query cache keys work?**

Arrays that uniquely identify a query, e.g. ['user', id]. Changing the key triggers a new fetch. Used for invalidation via queryClient.invalidateQueries.

Q9
9

Difference between global, server, and UI state.

UI state is local component state (open, hovered). Server state is remote data cached client-side. Global state is app-wide client data. Different tools fit each best.

Q1
00

How do you handle race conditions in data fetching?

AbortController to cancel stale requests, track request IDs, or use libraries (React Query) that handle it. Ignore results if a newer request has started.

Q1
01**What is Node.js?**

A JavaScript runtime built on V8 with an event-driven, non-blocking I/O model, ideal for I/O-heavy servers. Ships with a rich standard library and npm ecosystem.

Q1
02**Explain the Node.js event loop.**

Phases (timers, pending callbacks, idle, poll, check, close). Each phase processes its callback queue. Microtasks (Promises, process.nextTick) drain between phases.

Q1
03**Difference between process.nextTick and setImmediate.**

nextTick callbacks run before the next event loop phase (highest priority). setImmediate runs in the check phase after I/O. Overusing nextTick can starve I/O.

Q1
04**What are streams?**

Objects for reading/writing data in chunks: Readable, Writable, Duplex, Transform. Enable processing large data with low memory. Pipe them together for backpressure handling.

Q1
05**Explain the cluster module.**

Forks worker processes sharing the same port, allowing multi-core scaling. PM2 or the built-in cluster APIs manage worker lifecycle. Preferred over one big process.

Q1
06**What are Worker Threads?**

True OS threads for CPU-bound tasks that would otherwise block the event loop. Communicate via message-passing or SharedArrayBuffer.

Q1
07**What is middleware in Express?**

Functions (req, res, next) that run in order per request. Can inspect, modify, short-circuit or delegate. Used for logging, auth, parsing, error handling.

Q1
08**Difference between CommonJS and ES Modules.**

CJS uses require/module.exports, is sync, and dynamic. ESM uses import/export, is async, statically analyzable, and standard. Node supports both but ESM is the future.

**Q1
09**

How do you handle errors in async Node code?

try/catch with async/await, .catch on promises, and centralized Express error middleware (err, req, res, next). Always emit 'error' events on streams and EventEmitters.

**Q1
10**

What are Buffers?

Fixed-length raw memory allocations outside V8 heap, ideal for binary data (file I/O, network). Convert to strings with `buffer.toString('utf-8')`.

**Q1
11**

What is npm vs npx vs pnpm/bun/yarn?

npm is the default package manager. npx runs binaries from packages without installing globally. pnpm/yarn/bun are faster alternatives with different lockfiles and features.

**Q1
12**

How do you protect a Node server from crashes?

Use a process manager (PM2), handle `uncaughtException/unhandledRejection` to log and exit, validate inputs, and set reasonable timeouts. Never swallow errors silently.

**Q1
13**

What are environment variables and how to manage them?

Config values stored outside code via `process.env`. Load from `.env` with `dotenv` locally; provide via platform secrets in production. Never commit secrets.

**Q1
14**

Explain the difference between blocking and non-blocking I/O.

Blocking waits for the operation to complete before returning. Non-blocking returns immediately and delivers the result via callback/promise/event, allowing the thread to serve other requests.

**Q1
15**

How do you scale a Node.js application?

Vertically via clustering/worker threads; horizontally with a load balancer across many instances. Offload heavy work to queues (BullMQ), cache with Redis, and use CDN for static assets.

Q1
16**What is REST?**

An architectural style using HTTP verbs on resources identified by URLs, stateless, cacheable, and layered. Responses commonly in JSON.

Q1
17**Explain HTTP methods and idempotency.**

GET/HEAD read (idempotent, safe). PUT replaces, DELETE removes (idempotent). POST creates (non-idempotent). PATCH partially updates (should be idempotent). Idempotent = same result on repeat.

Q1
18**Common HTTP status codes.**

200 OK, 201 Created, 204 No Content, 301/302 Redirect, 304 Not Modified, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 429 Too Many Requests, 500 Server Error, 502/503/504 Gateway.

Q1
19**REST vs GraphQL.**

REST exposes multiple endpoints returning fixed shapes. GraphQL exposes a single endpoint where clients query exactly the fields they need. GraphQL reduces over/under-fetching but adds complexity.

Q1
20**What is over-fetching and under-fetching?**

Over-fetching: endpoint returns more data than needed. Under-fetching: client must call multiple endpoints. GraphQL and BFF patterns address both.

Q1
21**Explain API versioning strategies.**

URL path (/v1/...), query param (?v=1), custom header (Accept-Version), or media type (Accept: application/vnd.app.v1+json). Path versioning is most common and easiest to cache.

Q1
22**What is pagination and its common styles?**

Offset/limit (simple but drifts on inserts), page-based, cursor-based (stable, scalable), and keyset. Cursor pagination is preferred for feeds and infinite scroll.

Q1
23**What is CORS?**

Cross-Origin Resource Sharing — browser mechanism that lets servers whitelist which origins can call them. Preflight OPTIONS requests happen for non-simple requests.

**Q1
24**

Explain rate limiting.

Limits requests per client per window to protect against abuse and ensure fairness. Common algorithms: token bucket, leaky bucket, fixed/ sliding window.

**Q1
25**

What are webhooks?

HTTP callbacks a service makes to your endpoint when an event occurs. Always verify signatures, respond quickly (queue heavy work), and be idempotent (events may repeat).

**Q1
26**

HATEOAS — what is it?

Hypermedia As The Engine Of Application State — responses embed links to related actions, letting clients navigate the API dynamically. Rarely fully adopted in practice.

**Q1
27**

What is the N+1 query problem in APIs?

A parent request triggers one query per child. Fix with joins, batching (DataLoader), or eager loading.

**Q1
28**

GraphQL resolvers and schema — what are they?

The schema declares types and operations. Resolvers are functions that produce the data for each field, running server-side and often calling databases/services.

**Q1
29**

gRPC vs REST.

gRPC uses HTTP/2 + Protocol Buffers, binary and strongly-typed, great for internal service-to-service. REST is text-based (JSON), universal, and browser-friendly.

**Q1
30**

How do you design idempotent APIs?

Use idempotency keys (client-supplied UUID) for POST, ensure PUT/DELETE use natural resource IDs, and dedupe writes on the server for a bounded time window.

Q1
31**What is a relational database?**

A database that stores data in tables with rows and columns, enforcing schema, relationships via foreign keys, and using SQL. Examples: PostgreSQL, MySQL, SQL Server.

Q1
32**Explain the types of SQL joins.**

INNER (matching rows), LEFT/RIGHT OUTER (all rows from one side + matches), FULL OUTER (all rows), CROSS (Cartesian product), SELF (join a table to itself).

Q1
33**What are ACID properties?**

Atomicity (all-or-nothing), Consistency (valid state transitions), Isolation (concurrent txns don't interfere), Durability (committed data survives crashes).

Q1
34**What is normalization?**

Organizing tables to reduce redundancy and dependency. 1NF: atomic values; 2NF: no partial dependencies; 3NF: no transitive dependencies. Denormalize for read-heavy workloads.

Q1
35**What is an index?**

A data structure (B-tree, hash) that speeds up lookups on specific columns. Trade: faster reads, slower writes and extra storage. Avoid over-indexing.

Q1
36**Clustered vs non-clustered index.**

Clustered stores table rows in the index order (one per table). Non-clustered stores pointers to rows (many per table). PostgreSQL uses only non-clustered.

Q1
37**What is a transaction?**

A unit of work executed atomically. BEGIN, COMMIT, or ROLLBACK. Enables safe multi-step operations.

Q1
38**Explain isolation levels.**

Read Uncommitted, Read Committed, Repeatable Read, Serializable — from least to most strict. Higher levels avoid dirty/non-repeatable/phantom reads but reduce concurrency.

**Q1
39**

Difference between WHERE and HAVING.

WHERE filters rows before aggregation. HAVING filters groups after GROUP BY aggregation.

**Q1
40**

What is a foreign key?

A column referencing a primary key in another table, enforcing referential integrity. Supports ON DELETE/UPDATE actions (CASCADE, SET NULL, RESTRICT).

**Q1
41**

What is a stored procedure?

Precompiled SQL routine stored in the database, callable by clients. Reduces round trips and centralizes logic but can complicate versioning.

**Q1
42**

Explain SQL injection and how to prevent it.

Attacker manipulates a query by injecting SQL through unvalidated input. Prevent with parameterized queries/prepared statements, ORM binding, and least-privilege DB users.

**Q1
43**

What is EXPLAIN/EXPLAIN ANALYZE?

A command that shows the query planner's execution strategy, including which indexes are used and estimated costs. Essential for optimization.

**Q1
44**

What are window functions?

Functions like ROW_NUMBER(), RANK(), SUM() OVER (...) that compute values across a set of rows without collapsing them via GROUP BY. Powerful for rankings, running totals.

**Q1
45**

What is a view?

A named SELECT statement stored in the database. Simplifies complex queries and can enforce access control. Materialized views cache the result for fast reads.

Q1
46**What is NoSQL and when to use it?**

Non-relational databases (document, key-value, wide-column, graph) built for scale, flexible schema, or specific access patterns. Use when relational fit or scale is a poor match.

Q1
47**Types of NoSQL databases.**

Document (MongoDB), Key-Value (Redis, DynamoDB), Wide-Column (Cassandra), Graph (Neo4j). Each targets different access patterns.

Q1
48**What is MongoDB?**

A document store using BSON documents in collections. Flexible schema, rich queries, secondary indexes, aggregation pipeline. Scales horizontally via sharding.

Q1
49**What is Redis and its common use cases?**

In-memory data store used for caching, session stores, rate limiting, queues, pub/sub, leaderboards. Sub-millisecond latency with persistence options.

Q1
50**Difference between SQL and NoSQL.**

SQL: strict schema, joins, ACID, vertical scale. NoSQL: flexible schema, denormalized data, often eventual consistency, horizontal scale. Choose per use case; polyglot persistence is common.

Q1
51**Explain CAP theorem.**

In a distributed system you can guarantee only two of Consistency, Availability, Partition tolerance. Networks partition, so real choice is CP vs AP.

Q1
52**What is eventual consistency?**

All replicas converge to the same state given no new writes — used by AP systems (DynamoDB, Cassandra). Reads may be stale momentarily.

Q1
53**MongoDB aggregation pipeline — what is it?**

A sequence of stages (\$match, \$group, \$project, \$lookup, \$sort) that transform documents. Analogous to SQL SELECT/JOIN/GROUP BY chains.

Q1
54

What is sharding?

Horizontal partitioning of data across nodes by a shard key. Enables scaling beyond a single machine but complicates queries that cross shards.

Q1
55

Redis persistence options.

RDB snapshots at intervals, AOF (append-only file) logs every write, or both. Tune per durability vs performance needs.

11 Authentication & Security

15 questions

Q1
56

Authentication vs Authorization.

Authentication verifies WHO you are (login). Authorization decides WHAT you can do (permissions/roles). Both are required for most secured features.

Q1
57

What is a JWT?

JSON Web Token — signed (and optionally encrypted) token with header.payload.signature. Stateless: the server verifies via signature without a session lookup.

Q1
58

JWT security best practices.

Short expiry + refresh tokens, store access token in memory (not localStorage), use httpOnly secure cookies for refresh, sign with strong algorithm (RS256), rotate keys, and validate aud/iss/exp.

Q1
59

Explain OAuth 2.0.

An authorization framework letting users grant limited access to their resources on one service to another without sharing credentials. Common flows: Authorization Code (with PKCE), Client Credentials, Device Code.

Q1
60

What is OpenID Connect?

An identity layer on top of OAuth 2.0. Adds an ID token (JWT) with user identity claims so the client can authenticate the user, not just get an access token.

Q1
61

Cross-Site Scripting (XSS) — what and how to prevent?

Injection of malicious JS into a page. Prevent by output-escaping user content, sanitizing HTML (DOMPurify), avoiding innerHTML with untrusted input, and setting a strict Content Security Policy.

Q1
62

Cross-Site Request Forgery (CSRF) — what and how to prevent?

Attacker tricks the browser into sending authenticated requests. Prevent with SameSite cookies, CSRF tokens, and requiring custom headers on state-changing requests.

Q1
63

What is CORS and how do you enable it safely?

Browser policy restricting cross-origin requests. Enable it server-side with an explicit allowlist of origins, allowed methods, and headers — never reflect Origin blindly.

Q1
64

How should passwords be stored?

Never in plaintext. Use a slow adaptive hash (bcrypt, argon2, scrypt) with per-user salt. Enforce length/entropy requirements and check breach lists.

Q1
65

What is HTTPS/TLS?

Encrypted HTTP using TLS. Provides confidentiality, integrity, and server authentication. Use HSTS to enforce and pin modern cipher suites.

Q1
66

Session-based vs token-based auth.

Sessions store server-side state and use a cookie ID; easy to revoke. Tokens (JWT) are stateless and scale well but harder to revoke. Hybrid (refresh tokens + short access tokens) is common.

Q1
67

What is Content Security Policy?

HTTP header restricting sources of scripts, styles, images, etc. Powerful XSS mitigation. Ship in report-only mode first, then enforce.

Q1
68

What are common OWASP Top 10 risks?

Broken access control, cryptographic failures, injection, insecure design, misconfiguration, vulnerable components, auth failures, integrity failures, logging failures, SSRF.

Q1
69

What is SSRF and how to mitigate?

Server-Side Request Forgery — server fetches attacker-controlled URLs, hitting internal networks. Validate URLs, allowlist hosts, block private IP ranges, and disable redirects to untrusted hosts.

Q1
70

Explain rate limiting for auth endpoints.

Throttle login/signup/reset endpoints per IP + per account to slow brute-force and credential stuffing. Combine with CAPTCHA, MFA, and breach detection.

Q1
71**What is horizontal vs vertical scaling?**

Vertical: bigger machine (simple, but limited and single point of failure). Horizontal: more machines behind a load balancer (harder, but elastic and fault-tolerant).

Q1
72**What is a load balancer?**

A component distributing traffic across servers. Layer 4 (TCP) or Layer 7 (HTTP). Provides health checks, SSL termination, sticky sessions.

Q1
73**Explain caching layers.**

Browser cache, CDN, reverse proxy (Varnish), application cache (in-memory), distributed cache (Redis/Memcached), database cache. Cache closest to the consumer.

Q1
74**Cache invalidation strategies.**

TTL, write-through, write-back, write-around, explicit invalidation on write, and versioned keys. 'There are only two hard things: cache invalidation and naming.'

Q1
75**What is a CDN?**

Content Delivery Network — geographically distributed servers that cache static and increasingly dynamic content closer to users, reducing latency and origin load.

Q1
76**Message queues — why use them?**

Decouple producers and consumers, absorb traffic spikes, retry failures, and enable async work. Options: RabbitMQ, Kafka, SQS, BullMQ.

Q1
77**Kafka vs RabbitMQ.**

Kafka: log-based, high throughput, ordered per partition, ideal for streaming and event sourcing. RabbitMQ: broker-based, flexible routing, good for task queues and RPC.

Q1
78**Database replication.**

Copying data across replicas: primary-replica (writes to one, reads from many) or multi-primary. Improves read scale and availability.

Q1
79

What is sharding at the system level?

Partitioning data across databases by key. Enables scale beyond one machine but requires shard-aware queries and rebalancing strategy.

Q1
80

Explain eventual vs strong consistency.

Strong: reads always reflect the last write (harder, slower). Eventual: reads may lag but converge (faster, more available). Choose per feature.

Q1
81

What is a circuit breaker?

A pattern that stops calls to a failing service after a threshold, returning fast failures until the service recovers. Prevents cascading failures.

Q1
82

Idempotency in distributed systems — why?

Networks retry. Operations must produce the same result if repeated, avoided duplicated side effects. Use idempotency keys or upserts.

Q1
83

How to design a URL shortener?

Hash or base62 an auto-increment ID, store mapping in KV store, add caching at the edge, track click analytics async, and handle collisions and custom aliases.

Q1
84

Design considerations for a real-time chat app.

WebSockets for bi-directional messaging, message queue for fan-out, persistent store for history, presence service, sharding by room/user, and delivery acks.

Q1
85

What is the difference between monolith and microservices?

Monolith: single deployable, easier to develop initially, harder to scale teams. Microservices: independently deployable services communicating over the network — higher ops cost, better team autonomy at scale.

Q1
86**What is CI/CD?**

Continuous Integration merges and tests code frequently. Continuous Delivery/Deployment automates releasing that code to staging/production. Shortens feedback loops and reduces risk per change.

Q1
87**What is Docker?**

A platform to package applications and dependencies into portable containers running on a shared kernel. Ensures 'runs the same everywhere.'

Q1
88**Docker image vs container.**

Image is the immutable template (layers + metadata). Container is a running instance of an image with its own writable layer and process namespace.

Q1
89**What is Kubernetes?**

An orchestration platform for containers. Handles scheduling, scaling, self-healing, rolling updates, service discovery, and config/secret management.

Q1
90**Explain Pods, Deployments, Services in Kubernetes.**

Pod: smallest deployable unit (1+ containers sharing net/storage). Deployment: declarative pod management with rolling updates. Service: stable network endpoint for a set of pods.

Q1
91**Blue-green vs canary deployment.**

Blue-green switches all traffic from old to new environment instantly (fast rollback). Canary routes a small % to new version, gradually increasing (safer, more complex).

Q1
92**What is Infrastructure as Code?**

Managing infrastructure via versioned code (Terraform, CloudFormation, Pulumi). Enables reproducibility, review, and automation.

Q1
93**Explain the twelve-factor app methodology.**

Guidelines for portable cloud apps: codebase, dependencies, config, backing services, build/release/run, processes, port binding, concurrency, disposability, dev/prod parity, logs, admin processes.

**Q1
94**

What is a container registry?

A service that stores container images (Docker Hub, GHCR, ECR). CI pushes; deployment platforms pull. Supports tagging, signing, and vulnerability scanning.

**Q1
95**

Compare AWS S3, EC2, Lambda.

S3: object storage. EC2: virtual machines. Lambda: serverless functions billed per request/ms. Pick per compute model and cost profile.

**Q1
96**

What are secrets management best practices?

Never in git; use a secrets manager (AWS Secrets Manager, Vault, Doppler); inject via env at runtime; rotate regularly; scope access via least privilege.

**Q1
97**

What is observability (logs, metrics, traces)?

Logs record events, metrics quantify system state over time, traces follow a request across services. Together they enable debugging and SLO monitoring.

**Q1
98**

What is a service mesh?

Infrastructure layer (Istio, Linkerd) that manages service-to-service traffic — mTLS, retries, timeouts, observability — without touching application code.

**Q1
99**

Serverless — pros and cons.

Pros: no server ops, per-use billing, fast scaling. Cons: cold starts, vendor lock-in, execution limits, harder local dev and long-running work.

**Q2
00**

Blue/Green vs Rolling deployment.

Rolling replaces instances gradually with the new version (minimal extra capacity). Blue-green stands up a parallel env and swaps traffic (fast rollback, higher cost).

Q2
01**Unit vs integration vs end-to-end tests.**

Unit: single function/class in isolation, fast. Integration: multiple units together (API + DB). E2E: whole app through the UI. Follow the testing pyramid — many unit, some integration, few E2E.

Q2
02**What is TDD?**

Test-Driven Development: write a failing test, write minimal code to pass, refactor. Produces well-tested code and forces clear API design.

Q2
03**What is mocking?**

Replacing a real dependency with a controllable fake to isolate the unit under test. Overuse leads to tests coupled to implementation.

Q2
04**Difference between mocks, stubs, spies, fakes.**

Stub returns canned data. Mock verifies interactions. Spy wraps a real function to observe calls. Fake is a working lightweight implementation (in-memory DB).

Q2
05**What is code coverage and its limits?**

Percentage of code executed by tests. Useful signal but not sufficient — 100% coverage can still hide bugs. Focus on critical paths, edge cases, and behavior.

Q2
06**Popular JS testing tools.**

Jest, Vitest (unit), React Testing Library (component), Playwright/Cypress (E2E), MSW (network mocking), Supertest (HTTP APIs).

Q2
07**Testing library philosophy.**

Test behavior from the user's perspective, not implementation details. Query by role, label, text — not by class name.

Q2
08**What are snapshot tests and when to use them?**

Serialize output and compare against a stored snapshot. Best for stable UI/serialized data; risky for volatile output that leads to lazy snapshot updates.

**Q2
09**

How do you test async code?

Return the promise or use await; use `jest.useFakeTimers` for time; ensure assertions run (`expect.assertions(n)`); avoid arbitrary `setTimeout` waits — poll or await events.

**Q2
10**

What is flaky test and how to fix it?

A test that passes/fails inconsistently. Fix by removing timing assumptions, isolating shared state, mocking network/time, and controlling randomness.

15 Performance & Web Vitals

10 questions

Q2
11

What are Core Web Vitals?

Google's key UX metrics: LCP (loading — <2.5s), INP (interactivity — <200ms), CLS (visual stability — <0.1). Impact SEO and user satisfaction.

Q2
12

Explain Largest Contentful Paint.

Time to render the largest visible element (hero image, headline). Optimize with fast server, preloaded critical assets, no render-blocking resources, and image optimization.

Q2
13

What is Cumulative Layout Shift?

Sum of unexpected layout shifts. Cause: images without dimensions, ads/embeds injected late, web fonts causing FOIT/FOUT. Fix with width/height attrs, size reservations, font-display:swap.

Q2
14

Techniques to reduce JS bundle size.

Tree-shaking, code splitting, dynamic imports, replacing heavy libs (moment→date-fns/day.js), lazy-loading routes/components, and analyzing with a bundle visualizer.

Q2
15

What is lazy loading?

Deferring load until needed — images (loading='lazy'), routes (dynamic import), components (React.lazy). Improves initial load and reduces bandwidth.

Q2
16

How does HTTP/2 improve performance?

Multiplexed streams over one connection eliminate head-of-line blocking, header compression, and server push reduce latency vs HTTP/1.1.

Q2
17

What are service workers?

Background scripts intercepting network requests to enable caching, offline support, background sync, and push notifications. Foundation of PWAs.

Q2
18

Explain critical rendering path.

HTML → DOM, CSS → CSSOM → Render Tree → Layout → Paint. Minimize render-blocking resources, inline critical CSS, defer JS.

**Q2
19**

What is prefetching vs preloading?

preload: fetch a critical current-page resource ASAP. prefetch: low-priority fetch for likely future navigations. dns-prefetch and preconnect optimize connection setup.

**Q2
20**

How to optimize images?

Serve modern formats (AVIF/WebP), responsive srcset/sizes, appropriate dimensions, compress, lazy-load below-the-fold, and use a CDN with automatic transformation.

Q2
21**Merge vs rebase.**

Merge preserves history and creates a merge commit. Rebase rewrites commits on top of the target branch, giving a linear history. Never rebase shared branches.

Q2
22**What is git stash?**

Temporarily shelves uncommitted changes so you can switch context. Restore with git stash pop/apply.

Q2
23**How do you resolve merge conflicts?**

Edit conflicting files, mark resolutions, run git add, then continue the merge/rebase. Use tools like VS Code's merge editor; prefer small, frequent merges to avoid big conflicts.

Q2
24**What are common Git workflows?**

Trunk-based (short-lived branches merged to main), GitHub Flow (branch → PR → merge), Git Flow (main/develop/feature/release/hotfix). Trunk-based scales best for CD.

Q2
25**What is a fast-forward merge?**

When the target branch has no new commits since branching, Git simply moves the pointer forward — no merge commit. --no-ff forces a merge commit for auditability.

17 Behavioral & Scenario

10 questions

Q2
26

Tell me about a challenging bug you fixed.

Frame with STAR (Situation, Task, Action, Result). Emphasize how you narrowed it down (logs, bisecting, reproducing), the fix, and lessons learned (added tests, monitoring).

Q2
27

How do you handle disagreements with teammates?

Listen fully, focus on user/business outcomes, propose a small experiment or POC, disagree-and-commit if unresolved. Document decisions.

Q2
28

Describe a time you had to learn a new technology quickly.

Discuss the driver (project deadline), your learning approach (docs, small POC, pairing), how you validated understanding, and the outcome delivered.

Q2
29

How do you prioritize tasks?

Impact vs effort, deadlines, dependencies, and risk. Align with product/PM, communicate trade-offs, and re-plan as new information arrives.

Q2
30

Give an example of a technical trade-off you made.

Concrete situation — e.g. chose SSR over SPA for SEO, accepting more server cost — and how you measured the outcome.

Q2
31

Describe a production incident you handled.

Detection, mitigation (rollback / feature flag), root cause analysis, blameless postmortem, and preventative actions (tests, alerts, runbooks).

Q2
32

How do you keep code quality high?

Code reviews with constructive feedback, linters/formatters in CI, tests for critical paths, small PRs, pair programming for tricky areas.

Q2
33

How do you mentor junior developers?

Pair on real tasks, ask questions instead of dictating, review with explanations, share resources, and give autonomy on scoped work.

**Q2
34**

Tell me about a project you're most proud of.

Pick one with measurable impact. Explain your role, technical decisions, challenges, and how you'd approach it differently today.

**Q2
35**

Where do you see full-stack development going?

Discuss real trends: edge/serverless, TypeScript everywhere, AI-assisted dev, component-driven design systems, streaming SSR, and observability-first culture.
